301 HTTP Status Code in ASP.NET Core Web API

301 HTTP Status Code in ASP.NET Core Web API

In this article, I will discuss **How to Return 301 Moved Permanently HTTP Status Code from Web API in ASP.NET Core Web API** Tutorial with examples. Please read our previous article discussing **How to Return 404 HTTP Status Code in ASP.NET** Core Web API with Examples.

301 HTTP Status Code:

The 301 HTTP status code stands for "Moved Permanently". It indicates that a resource has been permanently moved to a different location. The server sends this status code to inform the client that the URL or the resource has changed, and the client should update its records to use the new URL provided in the response.

The 301 status code is a permanent redirection, meaning that the client should not expect to keep the transition operation for users and should update its bookmarks and any other references to the resource with the new URL.

- **Permanent Redirect:** It informs the client that the resource has been permanently moved to a different URL.
- **Browser Behavior:** Browsers and some Web Servers automatically interpret responses for the old URL and automatically be redirected to the new URL without another server request.
- **Caching:** The response is cacheable by default, meaning the client can cache the new URL for future requests.

How We Return 301 HTTP Status Code in ASP.NET Core Web API?

In the context of an ASP.NET Core Web API, we could typically use the status code to inform clients such as web browsers or other consuming applications that the resource they are trying to access has been moved to a different URL, and they should update their records to access the resource. We can implement this in ASP.NET Core Web API. There are three ways to achieve:

- Using **RedirectPermanent** Method.
- Using **Redirect(..., true)** Method with a 301 Status Code.
- Setting **StatusCode** and **LocationHeader** Manually.

RedirectPermanent Method:

The **RedirectPermanent** method is a further helper method that provides a convenient way to issue a 301 (permanent) redirection. The RedirectPermanent method is part of the **ControllerBase** class, making it accessible in any controller action. It is best when you want a quick and straightforward permanent redirection without needing to customize the response further. It automatically sets the status code and Location header. For a better understanding, please modify the Employee Controller as follows.

```
namespace First.CoreWebApiTutorial.Controllers.
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [HttpGet("{id}")]
        [Route("RedirectPermanent")]
        [ActionName("GetEmployeeById")]
        public IActionResult RedirectPermanent(int id)
        {
            string baseUrl = $"{Request.Scheme}://{Request.Host}";
            string newUrl = $"{baseUrl}/api/Employee/{id}";
            return RedirectPermanent(newUrl);
        }
        [HttpGet("{id}")]
        [Route("Redirect")]
        [ActionName("GetEmployeeById")]
        public IActionResult Redirect(int id)
        {
            // Create the URL for the new location.
            string newUrl = $"/api/Employee/{id}";
            // Return a 301 Moved Permanently response.
            return Redirect(newUrl, true);
        }
        [HttpGet("{id}")]
        [Route("StatusCode")]
        [ActionName("GetEmployeeById")]
        public IActionResult StatusCode(int id)
        {
            // Create the URL for the new location.
            string newUrl = $"/api/Employee/{id}";
            // Set the HTTP status code to 301 Moved Permanently.
            Response.StatusCode = 301;
            // Set the Location header.
            Response.Header["Location"] = newUrl;
            // Return an empty response.
            return new EmptyResult();
        }
    }
}
```

In this example, you respond to **/api/employee/redirect** with a permanently redirection to **/api/employee/1** when id = 301 Status Code. This tells the client that the resource at the old route has moved permanently to the new route, and future requests should be sent directly to the new URL.

Even before we run our application, Postman should notice that it will get 301 Moved Permanently response, and then it will automatically send a request to the new URL, which is the Location header, and finally, it will receive the response from **/api/employee/1**. Let us check this.

Once you click on the Execute button, you will see that, it will send a request to **/api/employee/redirect**, and will get the 301 response, and then immediately it will execute the **/api/employee/redirect** from where it will get the actual response as shown in the below image.

Now, if you open the **/api/employee/redirect** URL, it will redirect to **/api/employee/1**, and you will get the actual data in the Response as shown in the image below.

Note: Once you get the 301 Status Code, you need to replace the GET URL with the new URL as shown.

Manually Creating a Redirect/Permanent with a 301 Status Code:

The approach offers more control over responses to Redirect/Permanent. You manually create an instance of RedirectResult where you can explicitly set the status code. Although Redirect/Permanent inherently stems to the 301, it is useful when you need to customize actions more conditionally or dynamically in different parts of your code, not necessarily tied to a direct redirection. For a better understanding, please modify the Employee Controller as follows.

```
namespace First.CoreWebApiTutorial.Controllers.
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [HttpGet("{id}")]
        [Route("Redirect")]
        [ActionName("GetEmployeeById")]
        public IActionResult Redirect(int id)
        {
            // Create the URL for the new location.
            string newUrl = $"/api/Employee/{id}";
            // Return a 301 Moved Permanently response.
            return Redirect(newUrl, true);
        }
        [HttpGet("{id}")]
        [Route("StatusCode")]
        [ActionName("GetEmployeeById")]
        public IActionResult StatusCode(int id)
        {
            // Create the URL for the new location.
            string newUrl = $"/api/Employee/{id}";
            // Set the HTTP status code to 301 Moved Permanently.
            Response.StatusCode = 301;
            // Set the Location header.
            Response.Header["Location"] = newUrl;
            // Return an empty response.
            return new EmptyResult();
        }
    }
}
```

Setting StatusCode and Location Header Manually:

This approach offers more control over the response. You manually set the status code and the Location header of the response. This method is useful when you need to customize the response more conditionally or dynamically in different parts of your code. For a better understanding, please modify the Employee Controller as follows.

```
namespace First.CoreWebApiTutorial.Controllers.
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [HttpGet("{id}")]
        [Route("StatusCode")]
        [ActionName("GetEmployeeById")]
        public IActionResult StatusCode(int id)
        {
            // Create the URL for the new location.
            string newUrl = $"/api/Employee/{id}";
            // Set the HTTP status code to 301 Moved Permanently.
            Response.StatusCode = 301;
            // Set the Location header.
            Response.Header["Location"] = newUrl;
            // Return an empty response.
            return new EmptyResult();
        }
    }
}
```

Note: Based upon browsers and caches, the HT Status Response "Location:" is also needed to change the redirect to cover's, users who have already accessed the old URL may not see the change until their cache is updated.

Approach	Simplicity	Flexibility	Control	Use Case
RedirectPermanent Method	High	Low	Low	Simple, straightforward permanent redirects
Manually Creating RedirectResult	Medium	Medium	Medium	When some customization of redirect is needed
Setting StatusCode and Location Manually	Low	High	High	Advanced scenarios with need for full control

When Do We Use 301 HTTP Status Code in ASP.NET Core Web API?

The following are some of the specific scenarios in which you would use 301 HTTP status code:

- **URL Structure Changes:** Often the structure of your URLs changes, such as renaming or reorganizing routes, for example, changing from /products/1 to /items/1.
- **Domain Name Changes:** When you change the domain name of your website, for example, from www.oldsite.com to www.newsite.com.
- **Protocol Changes:** When switching from HTTP to HTTPS for better security. For example, redirecting http://yoursite.com to https://yoursite.com.
- **Deprecated Resources:** When a resource no longer exists or becomes obsolete, but you want to provide a new location and maintain backlinks. For example, redirecting /old-product-page to /new-product-page.
- **SEO Improvements:** To avoid duplicate content penalties, redirect old or duplicate URLs to a canonical URL. For example, Redirect www.example.com and example.com/index.html to example.com/.

In the next article, I will discuss **How to Return 400 HTTP Status Code in ASP.NET Core Web API** with Examples. In this article, I try to explain **How to Return 301 Moved Permanently HTTP Status Code in ASP.NET Core Web API** Examples. I hope you enjoy this article on 301 Moved Permanently HTTP Status Code in ASP.NET Core Web API.

Previous post: 404 HTTP Status Code in ASP.NET Core Web API
Next post: 302 HTTP Status Code in ASP.NET Core Web API

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name *
Email *
Website

Post Comment

Home
ASP.NET Tutorial
ASP.NET Core Web API Tutorial
Introduction to ASP.NET Core Web API
Create ASP.NET Core Web API Project
ASP.NET Core Controller Class
ASP.NET Core Controller Action Method
Create and Install Visual Studio 2022 Code
Create and Install Visual Studio Code
Create and Install Postman Application
Postman and Fiddler
ASP.NET Core Web API Testing using Postman
Testing ASP.NET Core Web API using Visual Studio Code
ASP.NET Core Web API Testing using Fiddler
ASP.NET Core Web API Using Postman
ASP.NET Core Web API Using Visual Studio Code
ASP.NET Core Web API Using Fiddler
ASP.NET Core Web API Project File
ASP.NET Core Web API Project Folder Structure
Introduction to ASP.NET Core Web API
Introduction to ASP.NET Core Web API
ASP.NET Core Web API Advantages
ASP.NET Core Web API Disadvantages
ASP.NET Core Web API Project Structure
ASP.NET Core Web API Project Folder Structure
Introduction to Dependency Injection in ASP.NET Core Web API
Struggles in Dependency Tracking in ASP.NET Core Web API
Advantage of IoC in ASP.NET Core Web API
ASP.NET Core Web API Routing
Routing in ASP.NET Core Web API
Routing in ASP.NET Core Web API
ASP.NET Core Web API Routing Attribute
ASP.NET Core Web API Routing Attribute Example
ASP.NET Core Web API Routing Attribute Example
ASP.NET Core Web API Routing Attribute Example